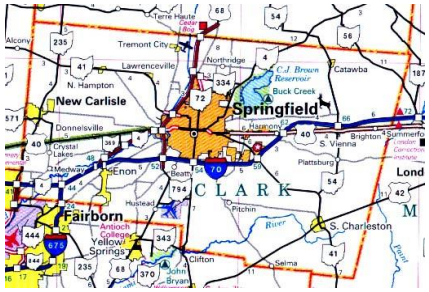
CSC3100: Graph I

Tianshu Yu

2024 Fall

Applications

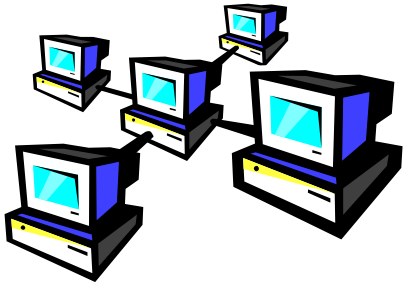
- Applications that involve not only a set of items, but also the connections between them



Maps

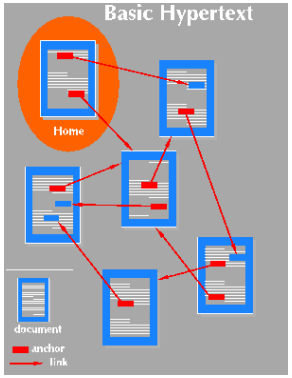


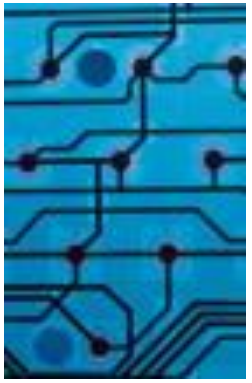
Schedules



Computer networks

Hypertext



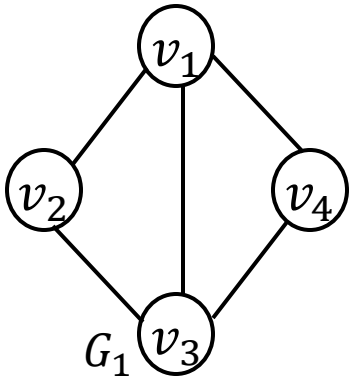


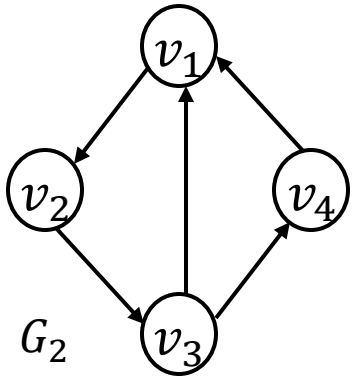
Circuits

Graph: definition

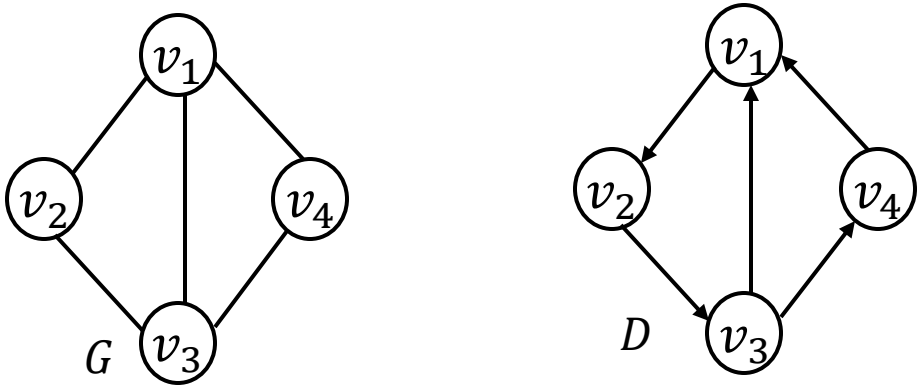
- A graph G is defined as a pair of (V, E) where:
  - V is the set of objects, each of which is called a node (or a vertex)
  - E is the set of edges, where each edge connects two nodes u and v

- Notations
  - n: the number of nodes, i.e., $|V|$
  - m: the number of edges, i.e., $|E|$





Undirected/directed graphs



- A graph is an undirected graph, if there is no order in an edge
  - We use (u, v) to represent an edge in an undirected graph
    - (u, v) and (v, u) are the same edge
  - In many social networks, e.g., Facebook

- A graph is a directed graph, if there is an order in an edge
  - We use $\langle u, v \rangle$ to represent an edge in a directed graph
    - $\langle u, v \rangle$ and $\langle v, u \rangle$ represent two different edges

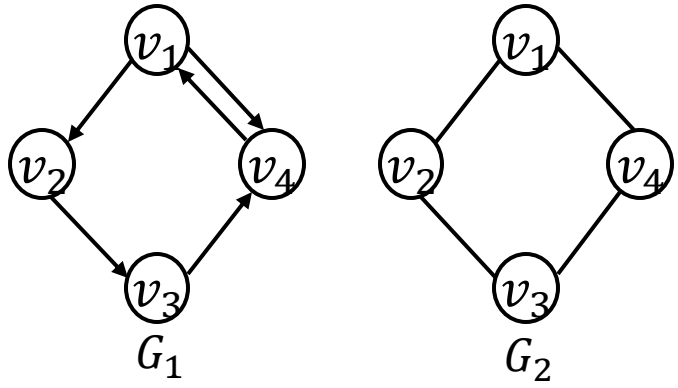
    In some social networks, e.g., Twitter

Terminologies

- Neighbor: v is called a neighbor of u, if there is an edge between v and u
  - In directed graphs, v is called the out-neighbor of u is there is an edge $\langle u, v \rangle$; v is called the in-neighbor of u is there is an edge $\langle v, u \rangle$
- Degree: the degree $d(v)$ of a node v is the number of neighbors of this node v
  - In directed graphs, the out-degree $d_{out}(v)$ of a node v is the number of out-neighbors of this node; the in-degree $d_{in}(v)$ of a node v is the number of in-neighbors of this node
- A graph is connected if there is a path from every vertex to every other vertex
  - A tree is a connected, acyclic "undirected" graph

An example

- In G_1, the degree of v_1 is 3, the out-degree of v_1 is 2, and the in-degree of v_1 is 1



- In G_2, the degree of v_3 is 2
- The number of edges in G_1 is 5
  - The degrees of v_1, v_2, v_3 and v_4 are 3, 2, 2, and 3, respectively
  - In G_1, $\sum_{v \in V} d(v) = 3 + 2 + 2 + 3 = 10 = 2 \cdot 5 = 2 \cdot m$
  - In G_1, $\sum_{v \in V} d_{out}(v) = 2 + 1 + 1 + 1 = 5 = m$, $\sum_{v \in V} d_{in}(v) = 1 + 1 + 1 + 2 = 5 = m$
- How about G_2?

Self-Edges, Connectedness

A self-edge a.k.a. a loop edge is of the form (u, u)

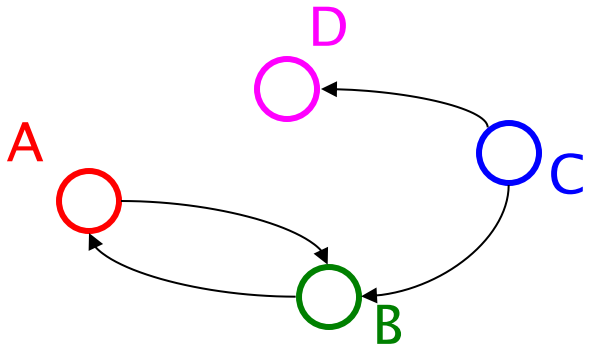
- The use/algorithm usually dictates if a graph has:
  - No self edges
  - Some self edges
  - All self edges

A node can have a(n) degree / in-degree / out-degree of zero

A graph does not have to be connected

- Even if every node has non-zero degree
- More discussion of this to come

More Notation



For a graph $G = (V, E)$:

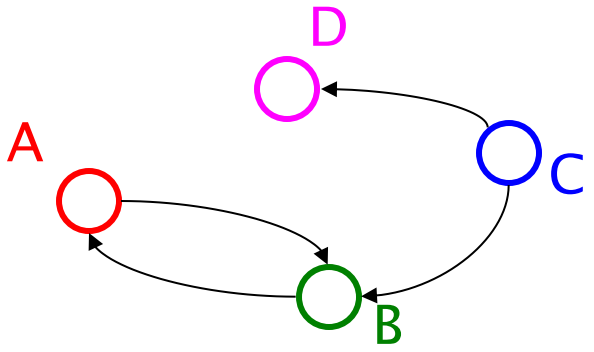
- $|V|$ is the number of vertices
- $|E|$ is the number of edges
  - Minimum?
  - Maximum for undirected?
  - Maximum for directed?

$V = \{A, B, C, D\}$
$E = \{(C, B), (A, B), (B, A), (C, D)\}$

If $(u, v) \in E$, then v is a neighbor of u (i.e., v is adjacent to u)

- Order matters for directed edges:
  u is not adjacent to v unless $(v, u) \in E$

More Notation



For a graph $G = (V, E)$:

- $|V|$ is the number of vertices
- $|E|$ is the number of edges
  - Minimum? 0
  - Maximum for undirected? $|V||V+1|/2 \in O(|V|^2)$
  - Maximum for directed? $|V|^2 \in O(|V|^2)$

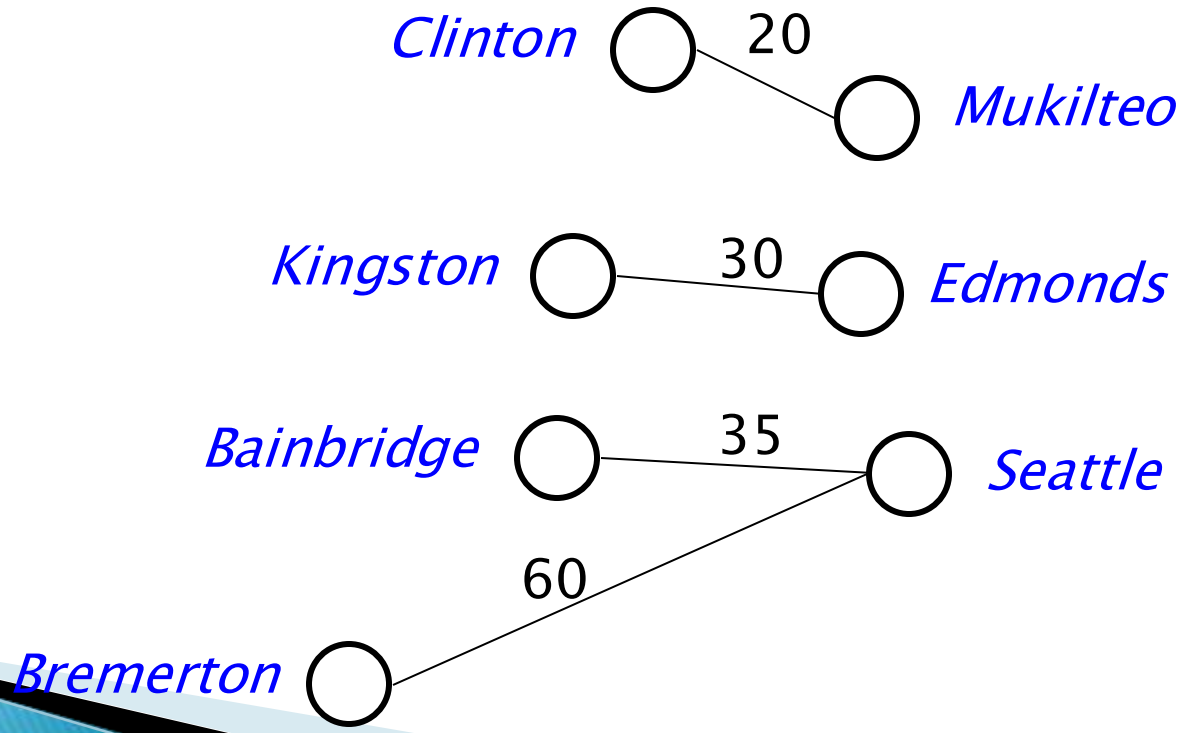
If $(u, v) \in E$, then v is a neighbor of u (i.e., v is adjacent to u)

- Order matters for directed edges:
  u is not adjacent to v unless $(v, u) \in E$

Weighted Graphs

In a weighted graph, each edge has a weight or cost

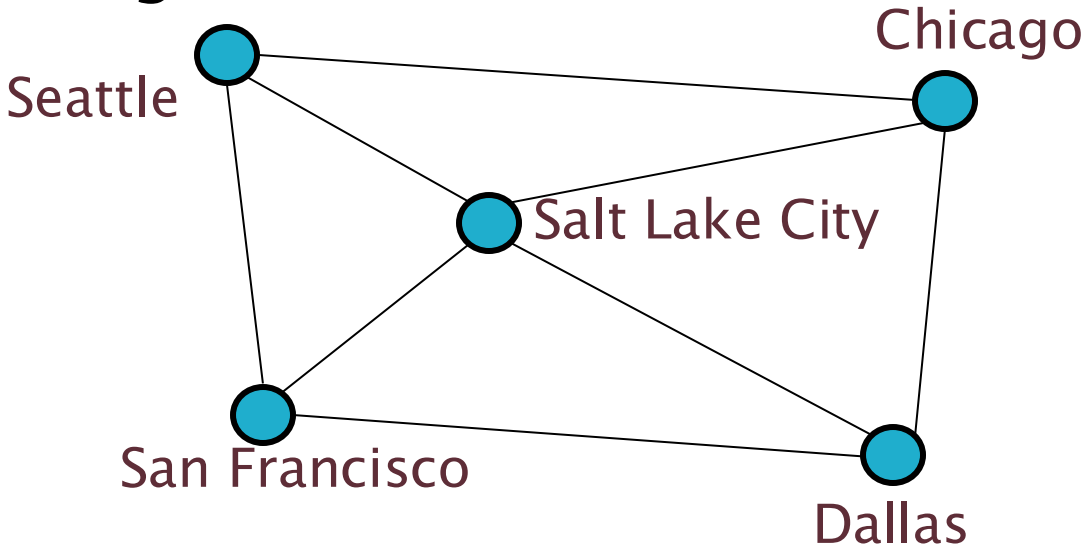
- Typically, numeric (ints, decimals, doubles, etc.)
- Orthogonal to whether graph is directed
- Some graphs allow negative weights; many do not



Paths and Cycles

We say "a path exists from v_0 to v_n" if there is a list of vertices $[v_0, v_1, ..., v_n]$ such that $(v_i, v_{i+1}) \in E$ for all $0 \leq i < n$.

A cycle is a path that begins and ends at the same node ($v_0 == v_n$)

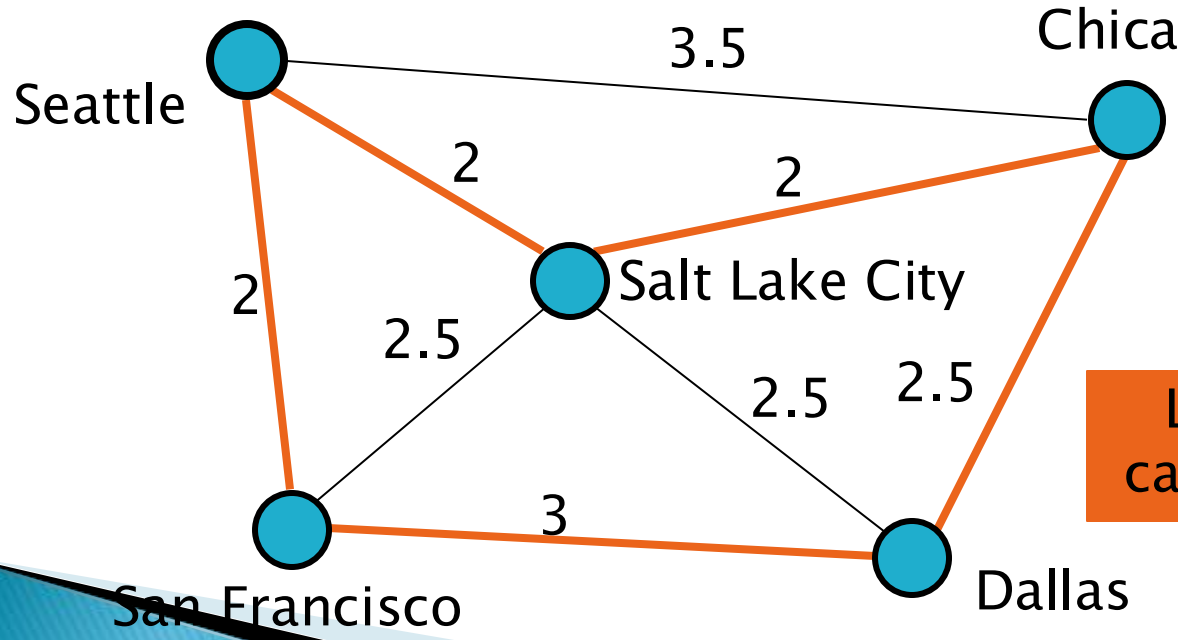


Example path (that also happens to be a cycle):
[Seattle, Salt Lake City, Chicago, Dallas, San Francisco, Seattle]

Path Length and Cost

Path length: Number of edges in a path
Path cost: Sum of the weights of each edge

Example where
P= [Seattle, Salt Lake City, Chicago, Dallas, San Francisco, Seattle]



length(P) = 5
cost(P) = 11.5

Length is sometimes called "unweighted cost"

Simple Paths and Cycles

A simple path repeats no vertices (except the first might be the last):

[Seattle, Salt Lake City, San Francisco, Dallas]

[Seattle, Salt Lake City, San Francisco, Dallas, Seattle]

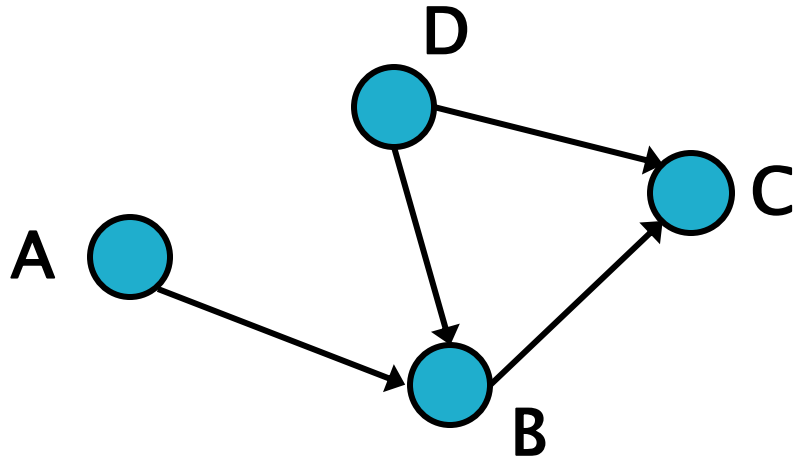
A cycle is a path that ends where it begins:

[Seattle, Salt Lake City, Seattle, Dallas, Seattle]

A simple cycle is a cycle and a simple path:

[Seattle, Salt Lake City, San Francisco, Dallas, Seattle]

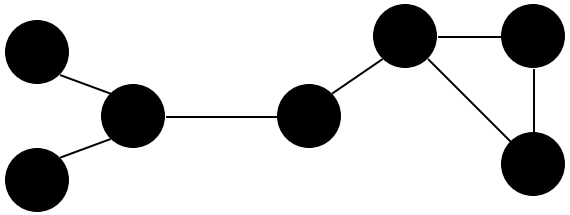
Example



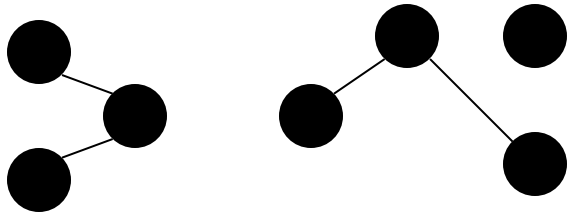
- Is there a path from A to D? No
- Does the graph contain any cycles? No

Undirected Graph Connectivity

An undirected graph is connected if for all pairs of vertices $u \neq v$, there exists a *path* from u to v

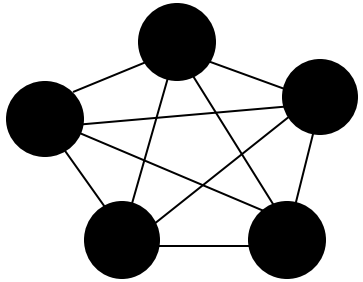


Connected graph



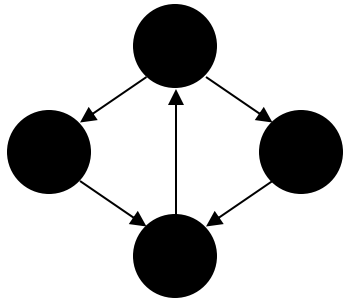
Disconnected graph

An undirected graph is complete, or fully connected, if for all pairs of vertices $u \neq v$ there exists an *edge* from u to v

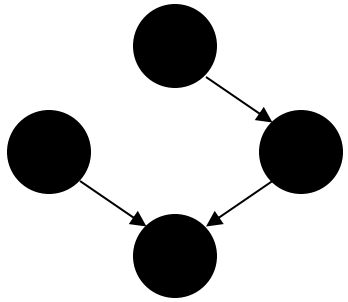


Directed Graph Connectivity

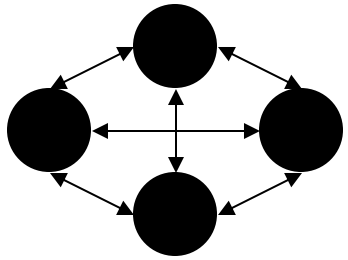
A directed graph is strongly connected if there is a path from every vertex to every other vertex



A directed graph is weakly connected if there is a path from every vertex to every other vertex *ignoring direction of edges*



A direct graph is complete or fully connected, if for all pairs of vertices $u \neq v$, there exists an *edge* from u to v



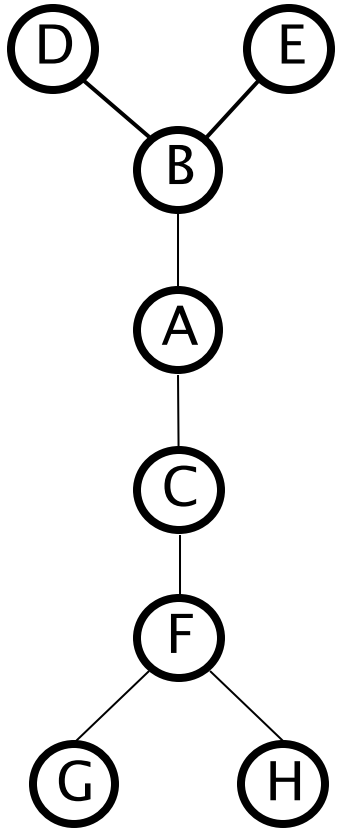
Trees as Graphs

When talking about graphs, we say a tree is a graph that is:

- undirected
- acyclic
- connected

All trees are graphs, but NOT all graphs are trees

How does this relate to the trees we know and "love"?



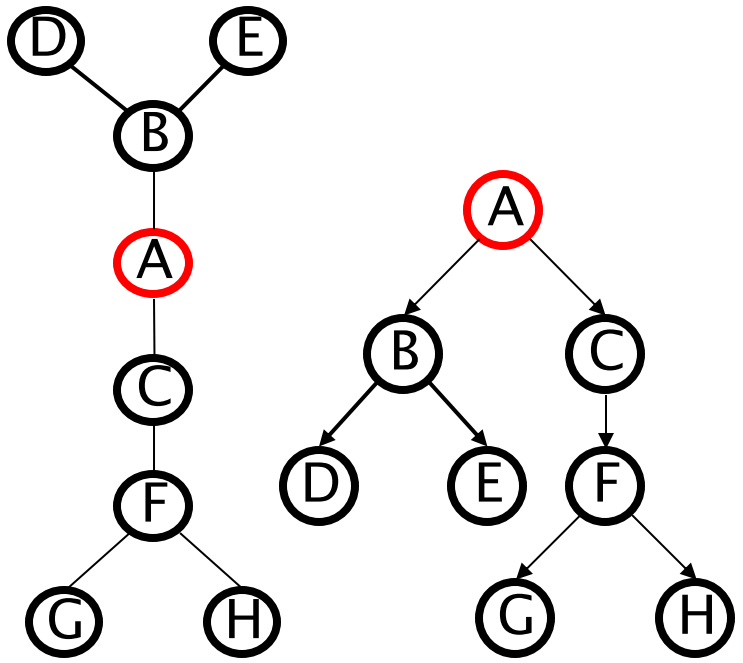
Rooted Trees

We are more accustomed to rooted trees where:

- We identify a unique root
- We think of edges as directed: parent to children

Picking a root gives a unique rooted tree

- The tree is simply drawn differently and with undirected edges



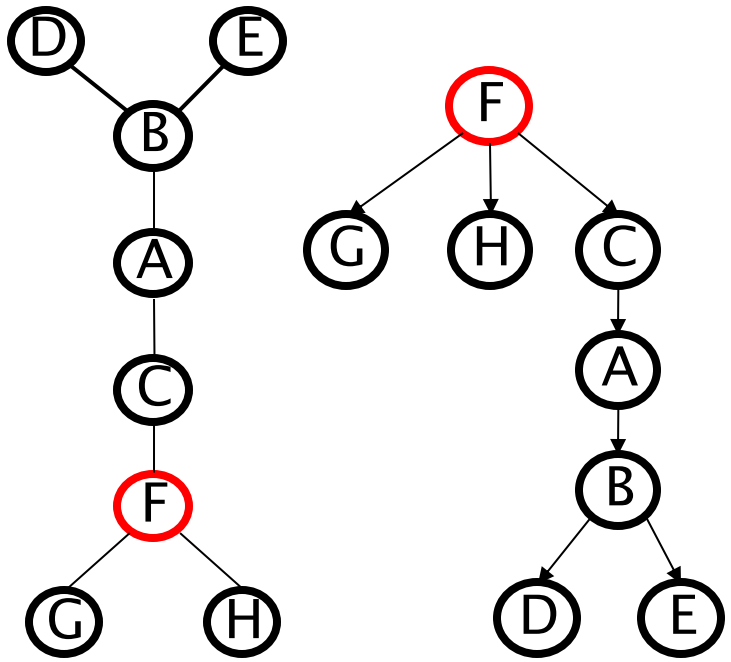
Rooted Trees

We are more accustomed to rooted trees where:

- We identify a unique root
- We think of edges as directed: parent to children

Picking a root gives a unique rooted tree

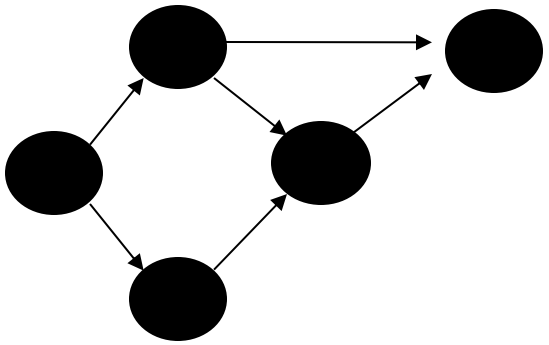
- The tree is simply drawn differently and with undirected edges



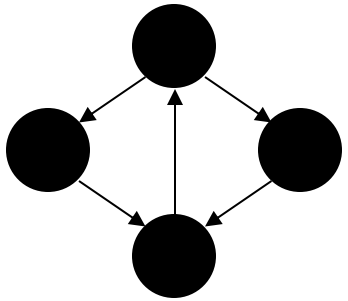
Directed Acyclic Graphs (DAGs)

A DAG is a directed graph with no directed cycles

- Every rooted directed tree is a DAG
- But not every DAG is a rooted directed tree



- Every DAG is a directed graph
- But not every directed graph is a DAG



Representation of graph

Graphs are often useful for lots of data and questions

- Example: "What's the lowest-cost path from x to y"

But we need a data structure that represents graphs

Which data structure is "best" can depend on:

- properties of the graph (e.g., dense versus sparse)
- the common queries about the graph ("is (u ,v) an edge?" vs "what are the neighbors of node u?")

We will discuss two standard graph representations

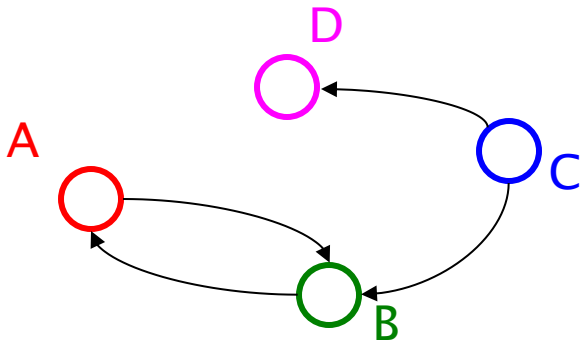
- Adjacency Matrix and Adjacency List
- Different trade-offs, particularly time versus space

Adjacency Matrix

Assign each node a number from 0 to |V|-1

A |V| x |V| matrix of Booleans (or 0 vs. 1)

- Then M[u][v] == true means there is an edge from u to v



	A	B	C	D
A	F	T	F	F
B	T	F	F	F
C	F	T	F	T
D	F	F	F	F

Adjacency Matrix Properties

Running time to:

- Get a vertex's out-edges:
- Get a vertex's in-edges:
- Decide if some edge exists:
- Insert an edge:
- Delete an edge:

Space requirements:

Best for sparse or dense graphs?

	A	B	C	D
A	F	T	F	F
B	T	F	F	F
C	F	T	F	T
D	F	F	F	F

Adjacency Matrix Properties

Running time to:

- Get a vertex's out-edges: $O(|V|)$
- Get a vertex's in-edges: $O(|V|)$
- Decide if some edge exists: $O(1)$
  - Insert an edge: $O(1)$
  - Delete an edge: $O(1)$

Space requirements:

$O(|V|^2)$

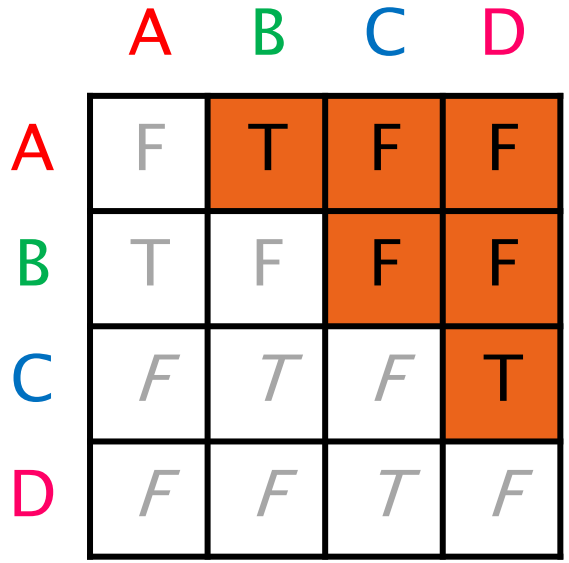
	A	B	C	D
A	F	T	F	F
B	T	F	F	F
C	F	T	F	T
D	F	F	F	F

Best for sparse or dense graphs? *dense*

Adjacency Matrix Properties

How will the adjacency matrix vary for an undirected graph?

- Will be symmetric about diagonal axis
- Matrix: Could we save space by using only about half the array?



- But how would you "get all neighbors"?

Adjacency Matrix Properties

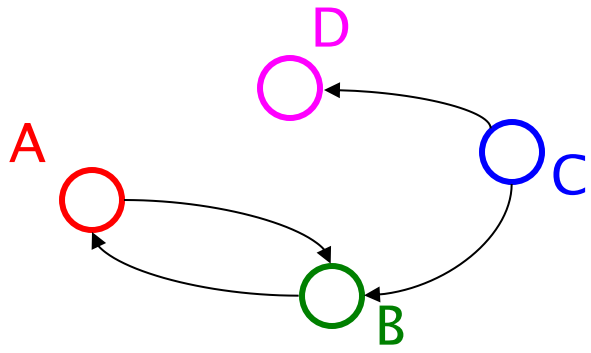
How can we adapt the representation for weighted graphs?

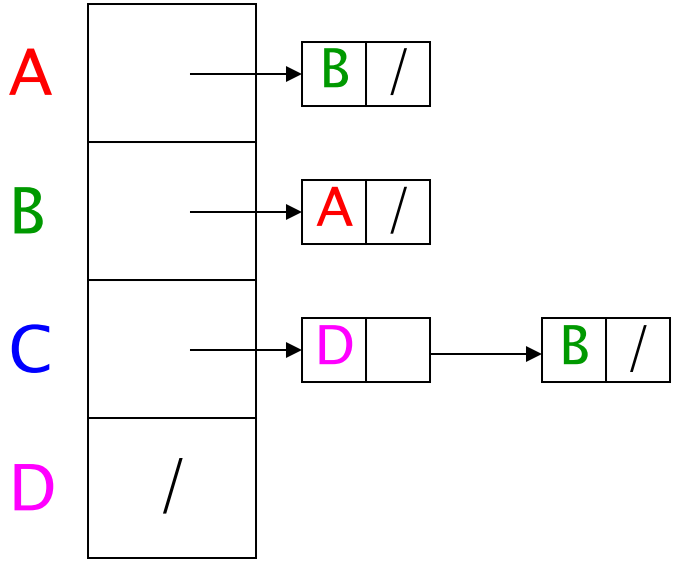
- Instead of Boolean, store a number in each cell
- Need some value to represent ‘not an edge’
  - 0, -1, or some other value based on how you are using the graph
  - Might need to be a separate field if no restrictions on weights

Adjacency List

Assign each node a number from 0 to |V|-1

- An array of length |V| in which each entry stores a list of all adjacent vertices (e.g., linked list)

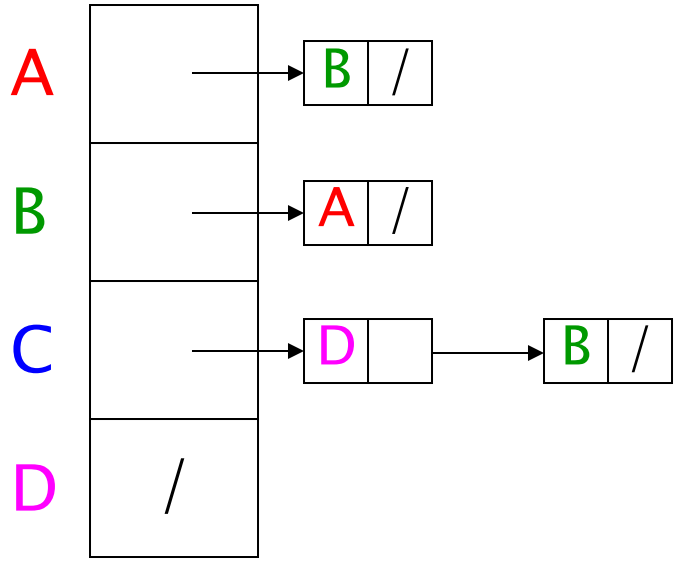




Adjacency List Properties

Running time to:

- Get a vertex's out-edges:
  - Get a vertex's in-edges:
  - Decide if some edge exists:
  - Insert an edge:
- Delete an edge:



Space requirements:

Best for sparse or dense graphs?

Adjacency List Properties

Running time to:

- Get a vertex's out-edges:
  $O(d)$ where d is out-degree of vertex
- Get a vertex's in-edges:
  $O(|E|)$ (could keep a second adjacency list for this!)
- Decide if some edge exists:
  $O(d)$ where d is out-degree of source
- Insert an edge:
  $O(1)$ (unless you need to check if it's already there)
- Delete an edge:
  $O(d)$ where d is out-degree of source

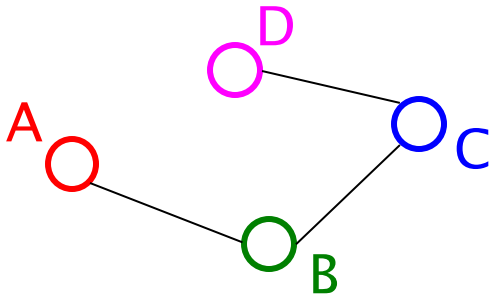
Space requirements: $O(|V|+|E|)$

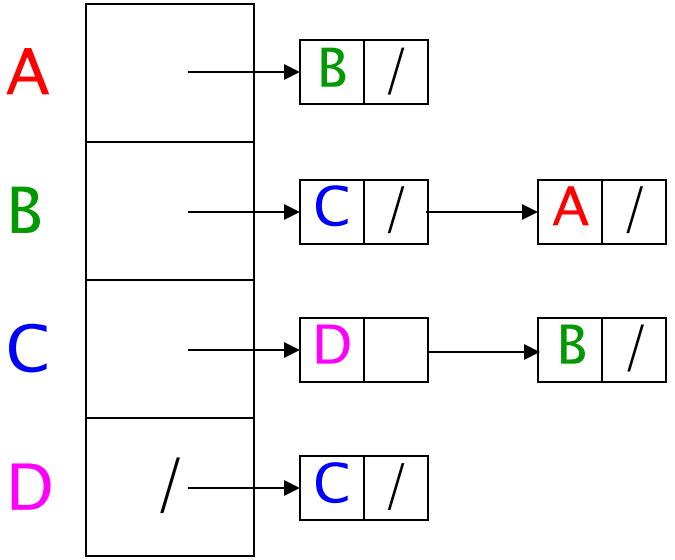
Best for sparse or dense graphs? *sparse*

Undirected Graphs

Adjacency lists also work well for undirected graphs with one caveat

- Put each edge in two lists to support efficient "get all neighbors"





Which is better?

Graphs are often sparse

- Streets form grids
- Airlines rarely fly to all cities

Adjacency lists should generally be your default choice

- Slower performance compensated by greater space savings

Adjacency list vs adjacent matrix

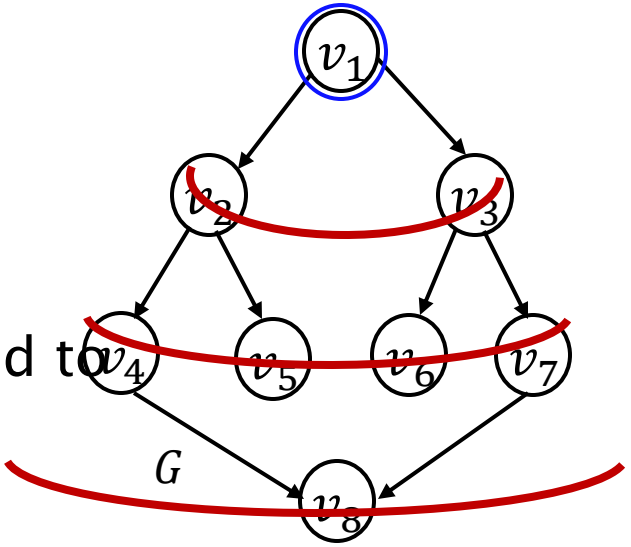
- Adjacency list:
  - Space: $O(n+m)$, save space if the graph is sparse, i.e., $m \ll n^2$
  - Check the existence of an edge (u, v): $O(k)$ time where k is the number of neighbors of v
  - Retrieve the neighbors of a node: $O(k)$ time
  - Add/delete a node: $O(n)$
  - Add/delete an edge: $O(k)$

- Adjacency matrix:
  - Space consumption: $O(n^2)$
  - Check the existence of an edge (u, v): $O(1)$ time
  - Retrieve the neighbors of a node: $O(n)$ time
  - Add/delete a node: $O(n^2)$, (create a new matrix)
  - Add/delete an edge: $O(1)$

Breadth-first search (BFS)

- Intuition of BFS
  - Given a source node s, always visit nodes that are closer to the source s first before visiting the others

- The result is not unique, if we do not define an order among out-going edges from a node
  - Possible results
    - $v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8$
    - $v_1, v_3, v_2, v_7, v_6, v_5, v_4, v_8$
  - If we impose an order by going from smaller id to larger id, then the result will be unique



BFS steps

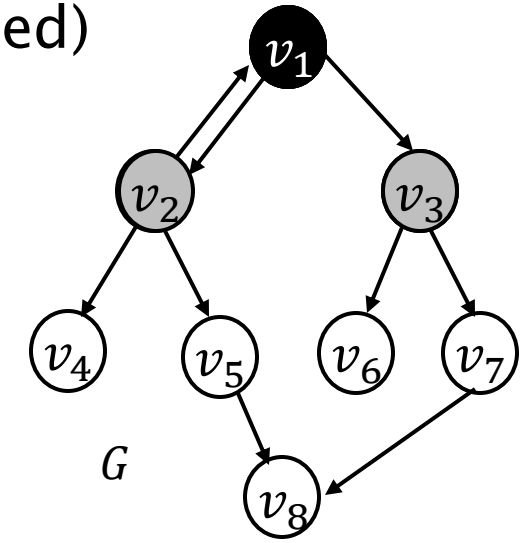
- At the beginning, color all nodes to be white
- Create a queue Q, enqueue the source s to Q, and color the source to be gray (meaning s is in the queue)
- Repeat the following until queue Q is empty
  - Dequeue from Q, let the node be v
  - For every out-neighbor u of v that is still white
    - Enqueue u into Q, and color u to gray (to indicate u is in queue)
  - Color v to be black (meaning v has finished)

- Example:
  - Assume the source is v_1

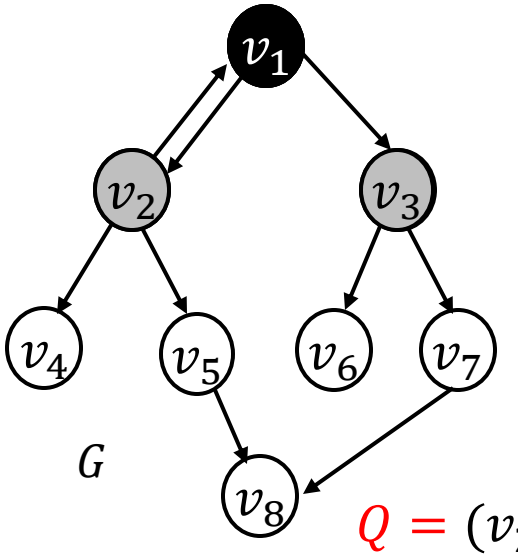
$Q = (v_1)$

After dequeuing v_1

$Q = (v_2, v_3)$

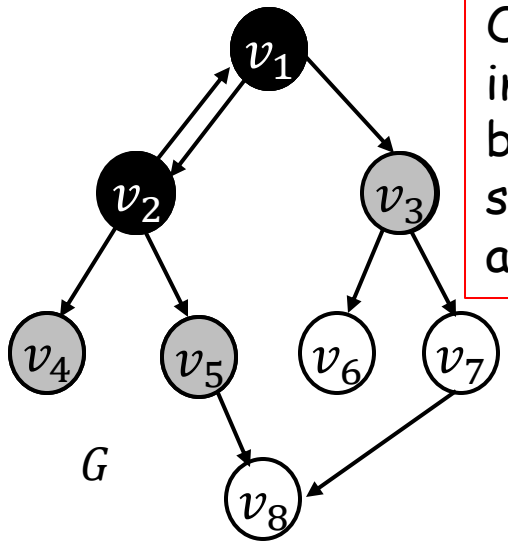


BFS: running example



$Q = (v_2, v_3)$

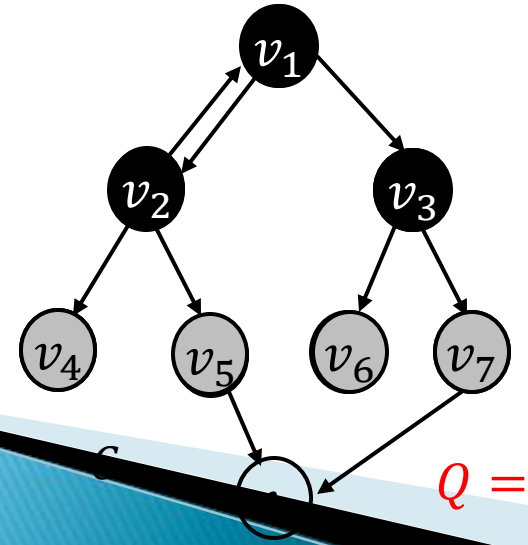
Dequeue from Q



Out-neighbor of v_2 includes v_1, v_4, v_5, but v_1 is not white, so only v_4 and v_5 are enqueued into Q

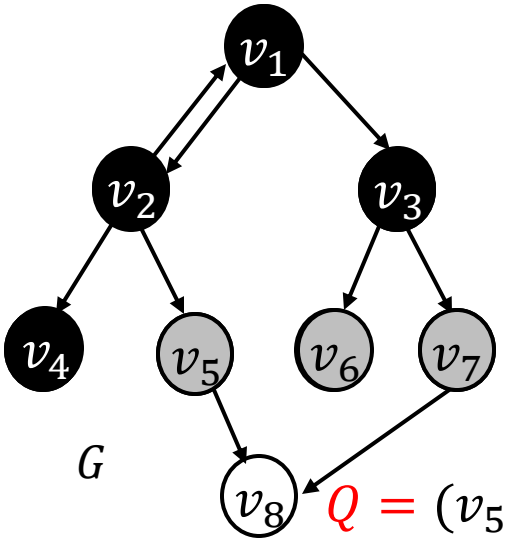
$Q = (v_3, v_4, v_5)$

Dequeue from Q



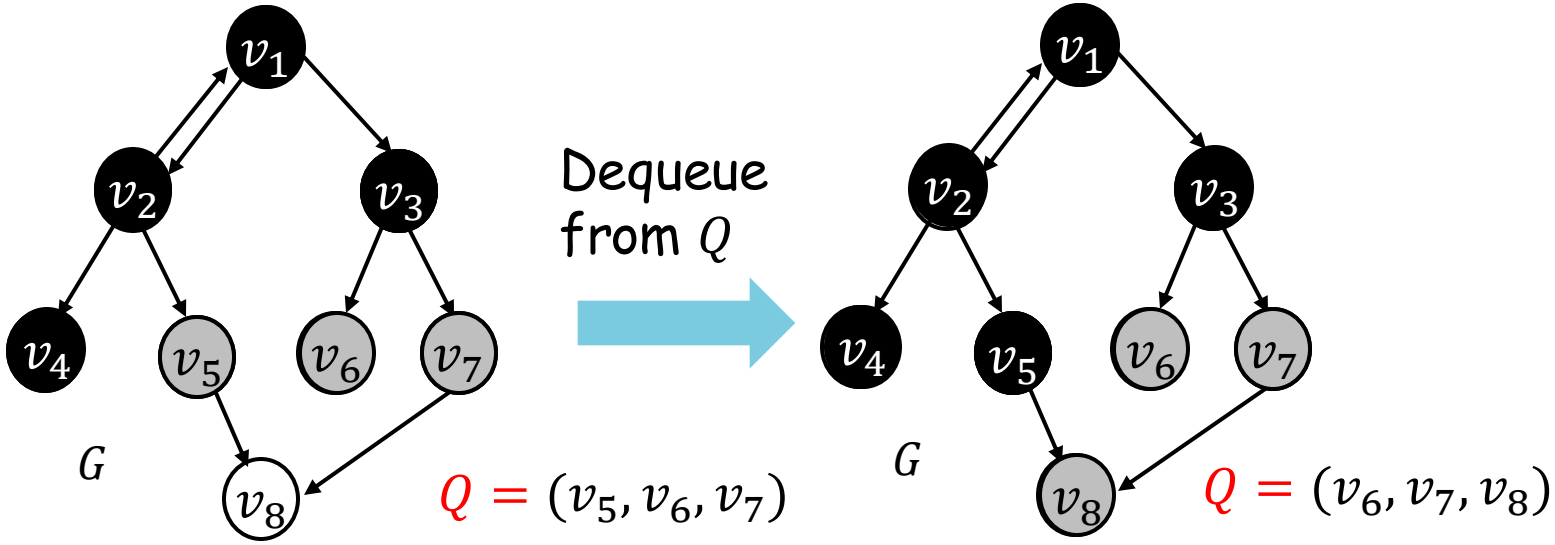
$Q = (v_4, v_5, v_6, v_7)$

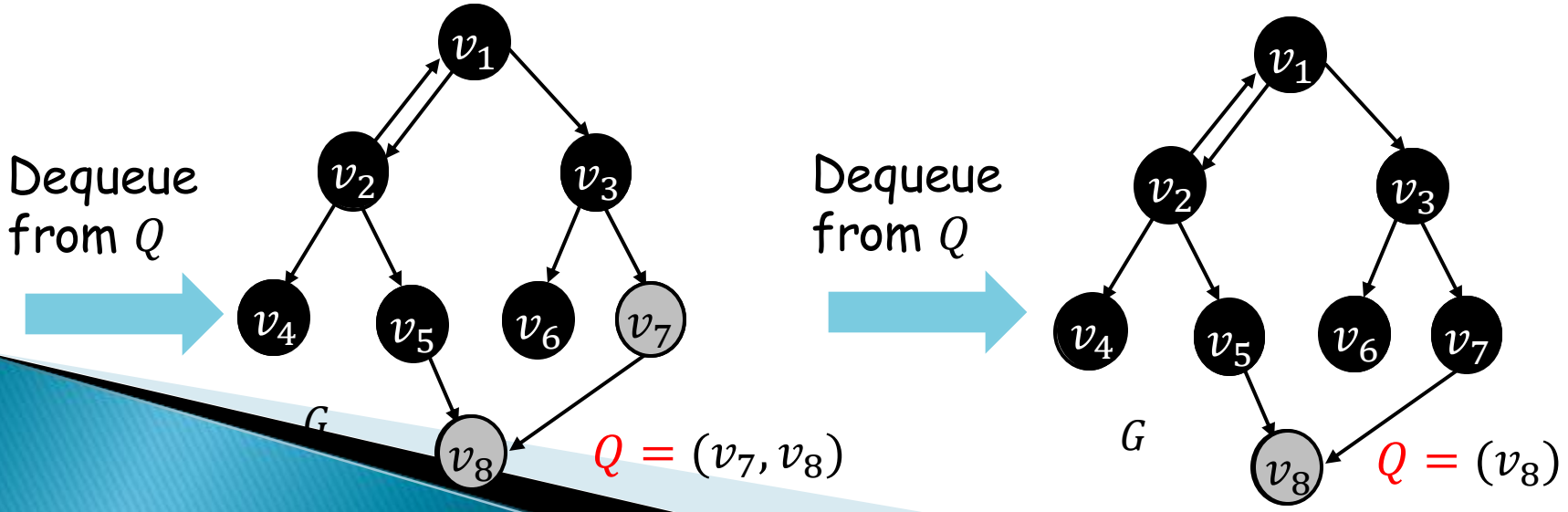
Dequeue from Q



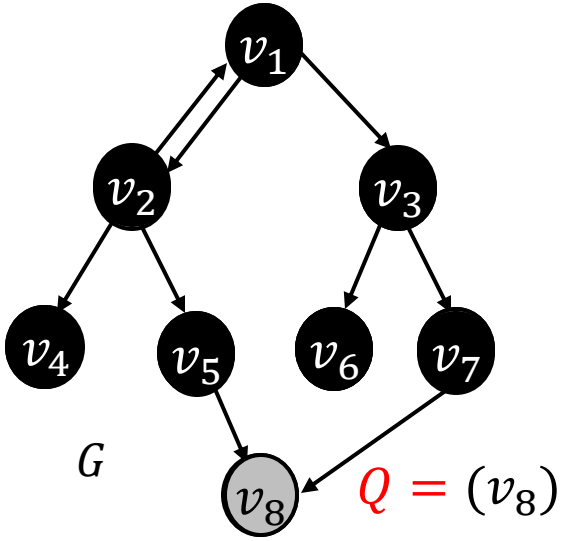
$Q = (v_5, v_6, v_7)$

BFS: running example

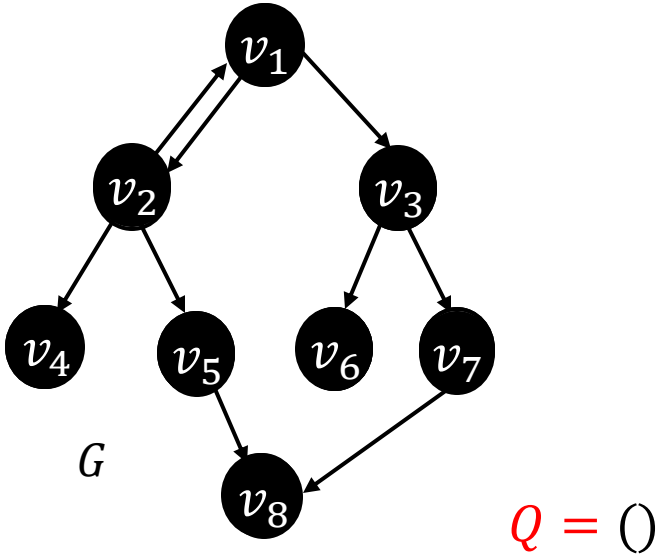




BFS: running example



Dequeue from Q

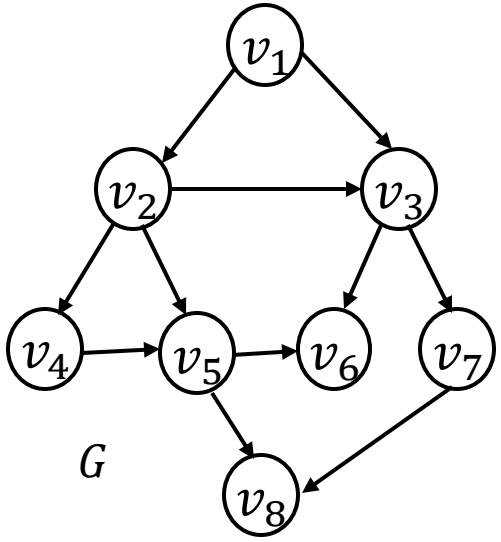


Now Q is empty

BFS finishes

Practice

- Given the following graph G, show the process of the BFS if the source is v_2
  - You may use 0 to denote the color to be white, 1 to denote the color to be gray, and 2 to denote the color to be black



BFS: implementation

Algorithm 1: ***BFS(G, s)***

```
1  color← allocate an array of size G.n, initialize with all zeros
2  // Use 0 : white, 1 : gray, and 2: black
3  Q ← an empty queue
4  Q.enqueue(s)
5  color[s] ← 1
6  while !Q.isEmpty()
7   v ← Q.dequeue
8   for u ∈ out-neighbor of v
9      if color[u]=0
10        Q.enqueue(u)
11        color[u]=1
12  color[v] ← 2
13  print v
14 free the array color if necessary
```

```
//adjacency matrix to store the graph
for u = 0 to G.n-1
  if G.adjmatrix[v][u]==1 and color[u]==0
     …

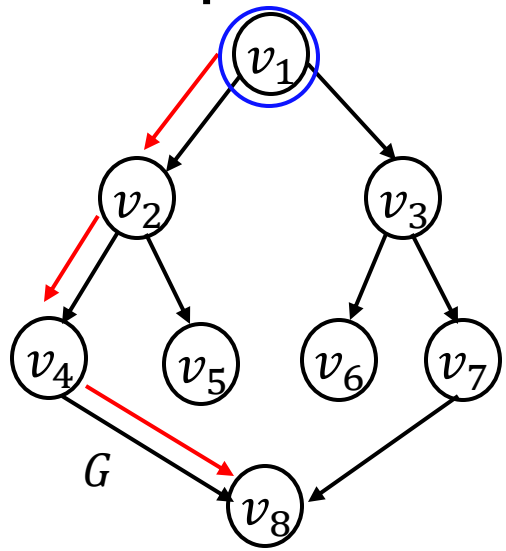
//adjacency list to store the graph
linkedlist_node = G[v].head.next
while linkedlist_node != G[v].tail
   u =linkedlist_node.element
   …
   linkedlist_node = linkedlist_node.next
```

BFS: time complexity

- When a node u is dequeued,
  - We examine all of its neighbors (check their color), enqueue them and color them to gray if they are white
  - After that, we color u as black
  - This incurs $c(1 + d_{out}(u))$ costs for node u where c is a constant (if we use adjacency list to represent the graph)
- Each node is dequeued at most once
  - Why? we enqueue a node at most once
  - If it is in the queue, its color is gray and we will not further enqueue it
- Therefore, the total running time with adjacency list representation is:
  - $\sum_{u \in V} c(1 + d_{out}(u)) = c(n + m) = O(n + m)$

Depth-first search (DFS)

- Going along one path until we cannot go further
- Imposing an order to make the traversal unique: from smaller id to larger id
  - Visiting order: $v_1, v_2, v_4, v_8, v_5, v_3, v_6, v_7$



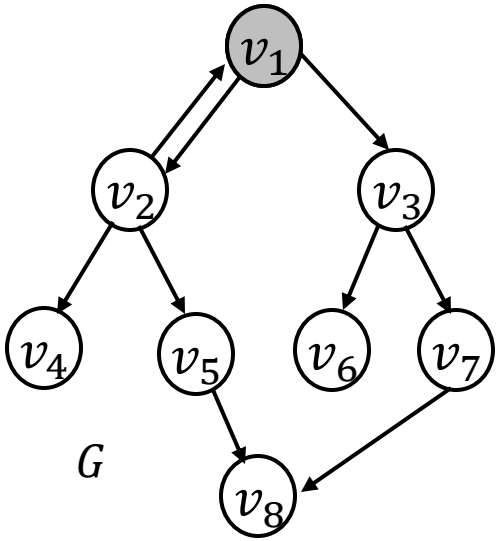
- We still focus on directed graph
  - Extension to undirected graph will be straightforward

Depth-first search (DFS)

- Initialization:
  - At the beginning, color all nodes to be white
  - Create a stack S, push the source s to S, and color the source to be gray (meaning s is in the stack)

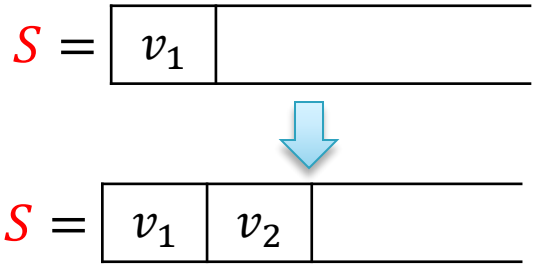
- Example:
  - Assume that v_1 is the source

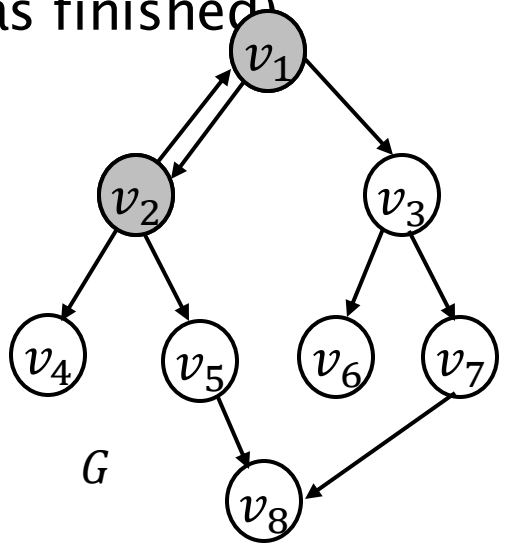
$S =$ | v_1 |



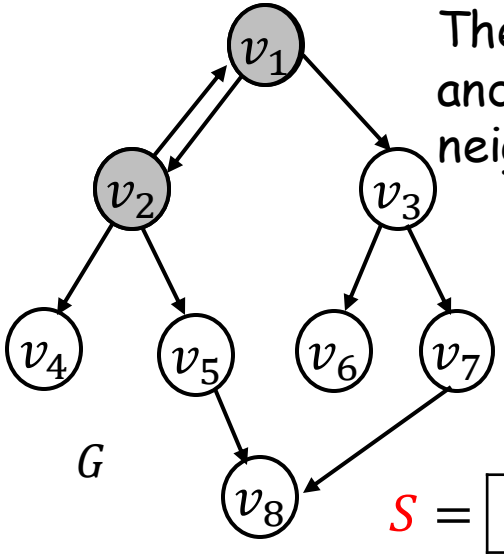
Depth-first search (DFS)

- Repeat the following until S is empty
  - Get the top node, denoted as v, on stack S, **do not** pop v
  - If v still has white out-neighbors
    - Let u be such a white out-neighbor of v
    - Push u to S, and color u to gray
  - Otherwise (v has no white out-neighbors)
    - Pop v and color it as black (meaning that v has finished)



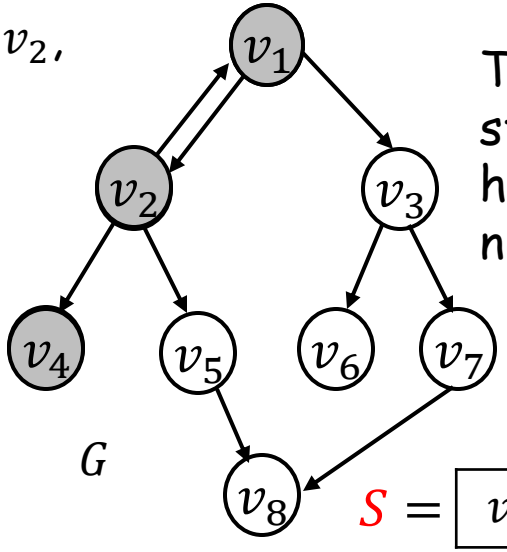


DFS: running example



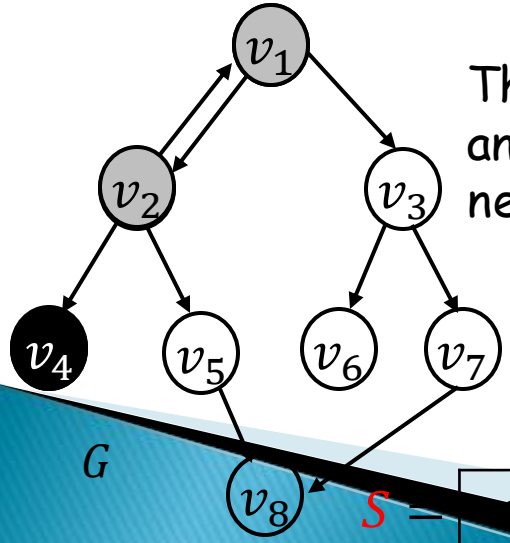
The top of the stack is v_2, and it has white out-neighbor v_4

$S =$ | v_1 | v_2 |



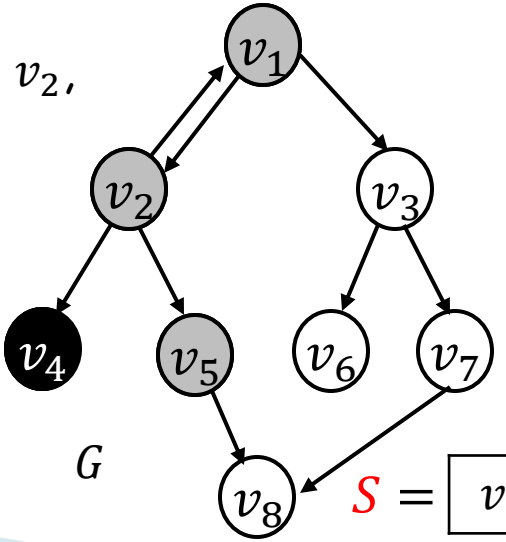
The top of the stack is v_4, and it has no white out-neighbor

$S =$ | v_1 | v_2 | v_4 |



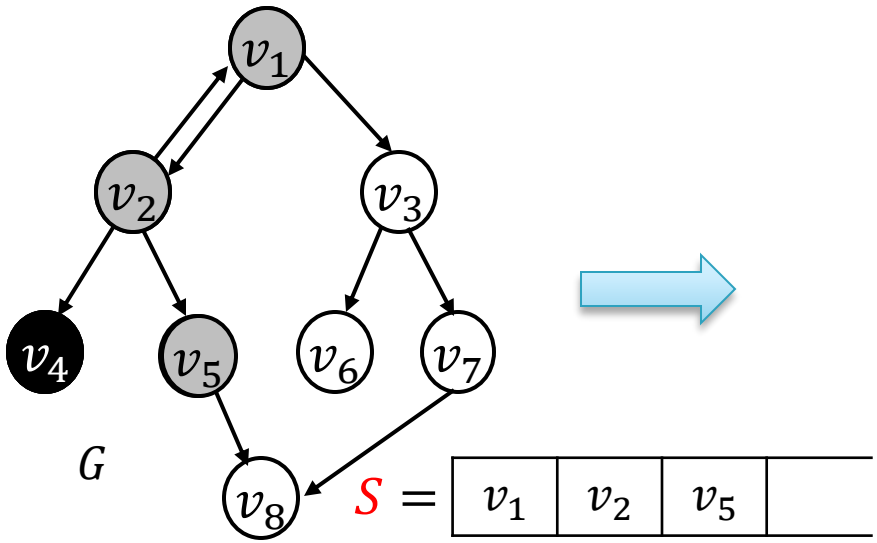
The top of the stack is v_2, and it has white out-neighbor v_5

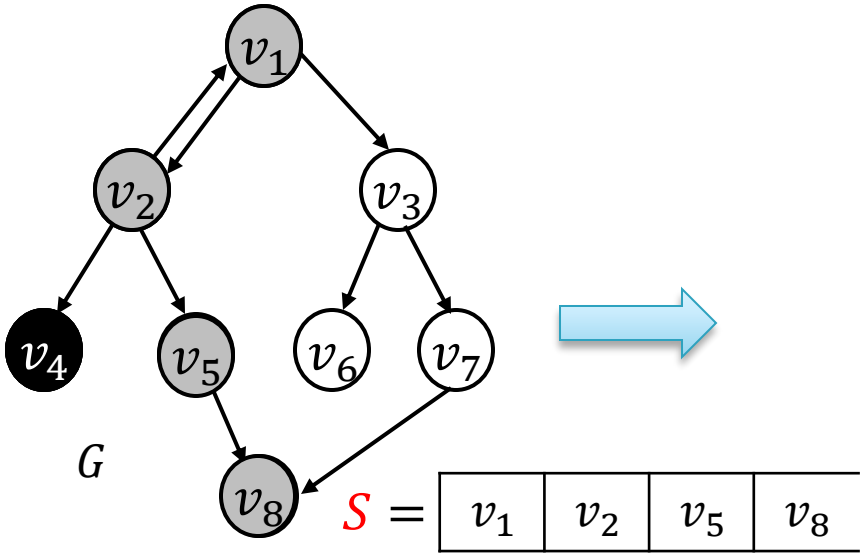
$S =$ | | v_2 |

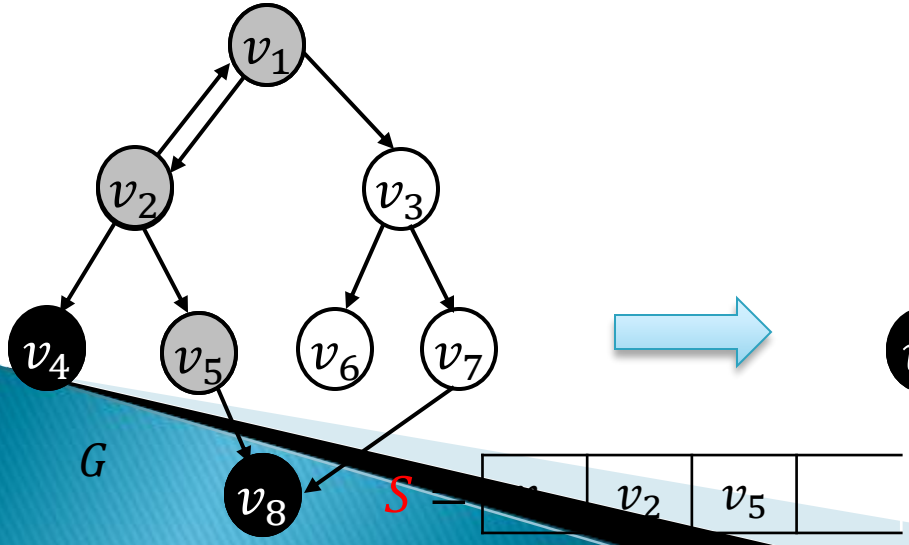


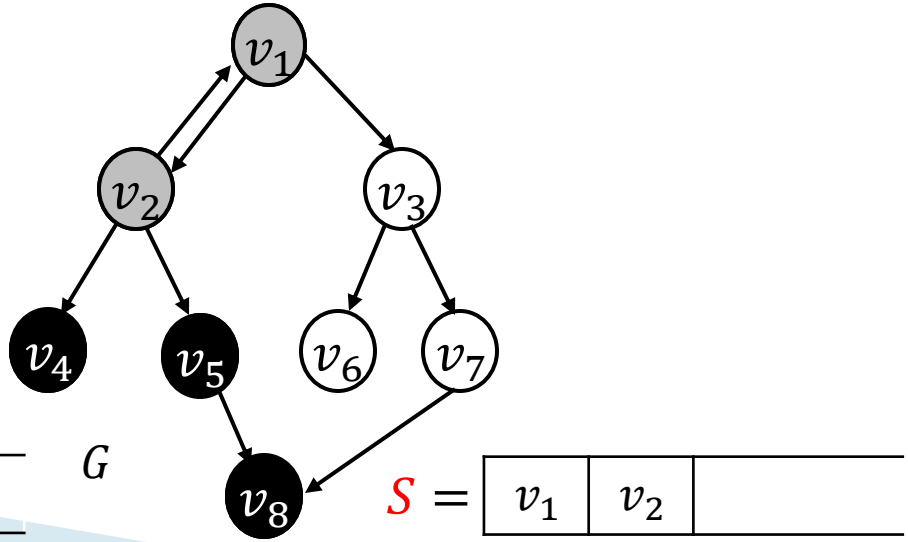
$S =$ | v_1 | v_2 | v_5 |

DFS: running example

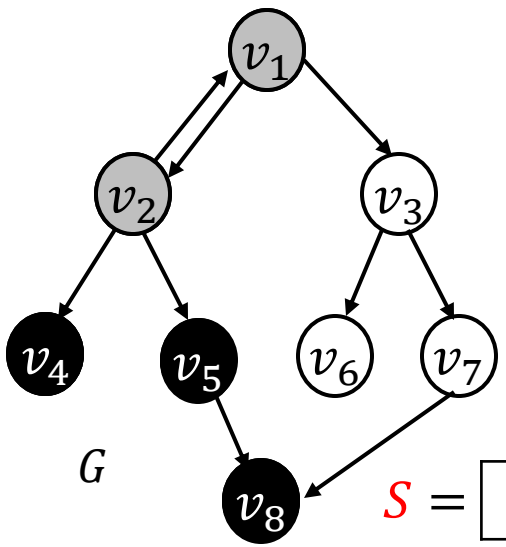




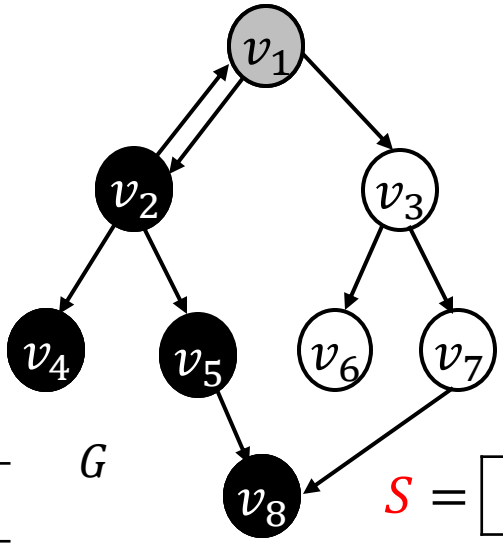




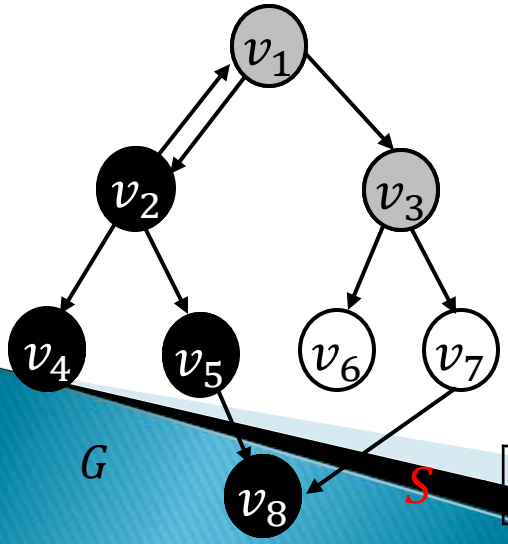
DFS: running example



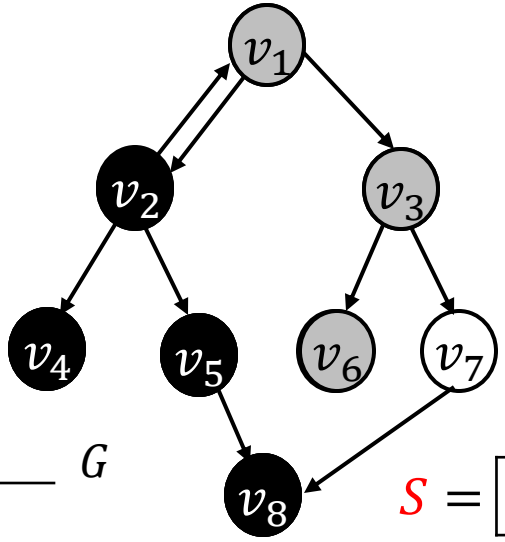
$S =$ | v_1 | v_2 |



$S =$ | v_1 |

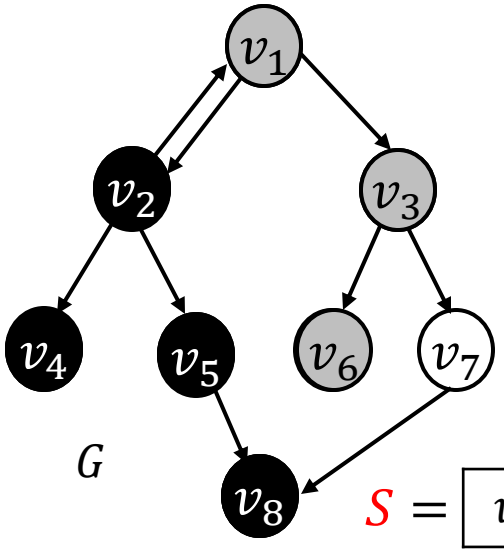


S | v_1 | v_3 |

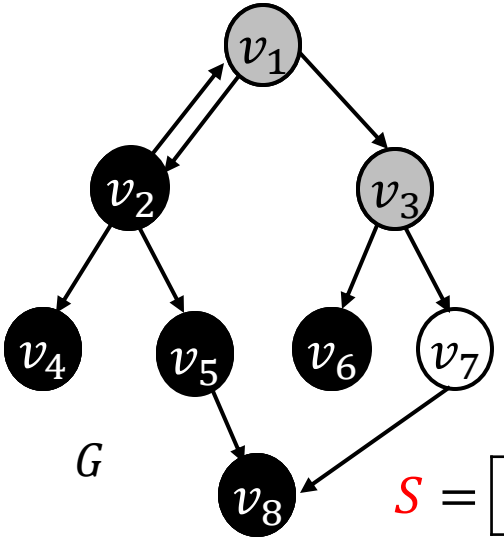


$S =$ | v_1 | v_3 | v_6 |

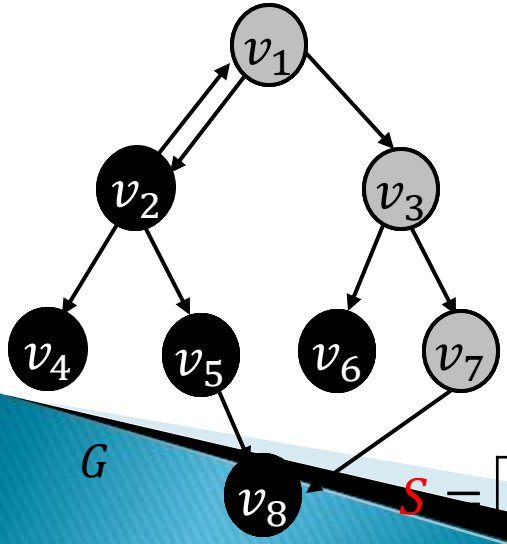
DFS: running example



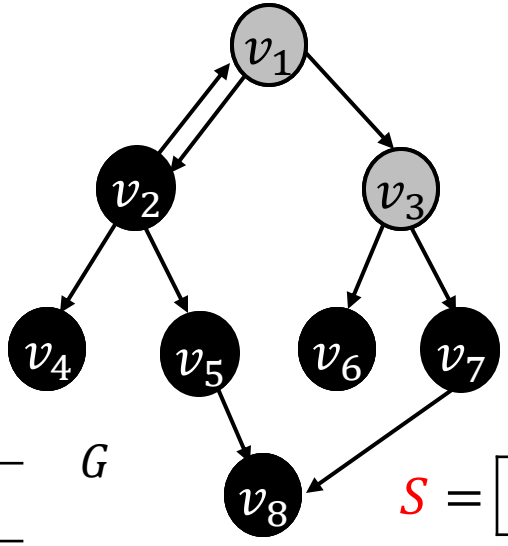
$S =$ | v_1 | v_3 | v_6 |



$S =$ | v_1 | v_3 |

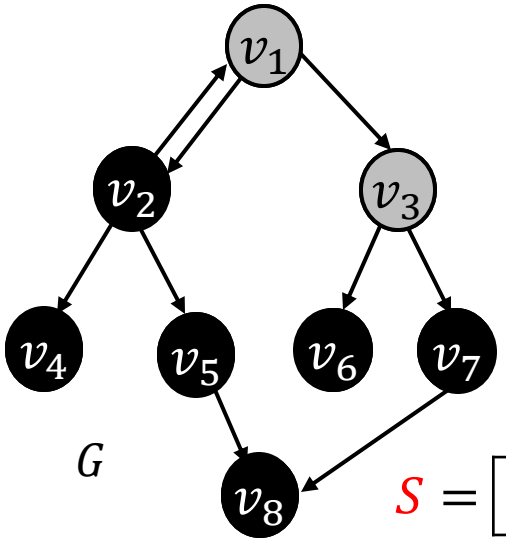


$S =$ | v_1 | v_3 | v_7 |



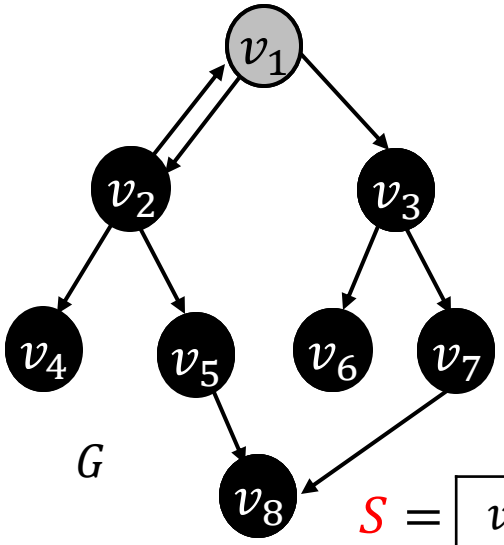
$S =$ | v_1 | v_3 |

DFS: running example



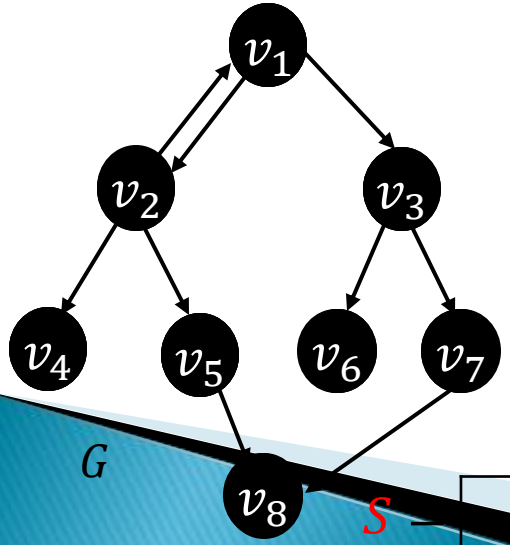
G

$S =$ | v_1 | v_3 |



G

$S =$ | v_1 |



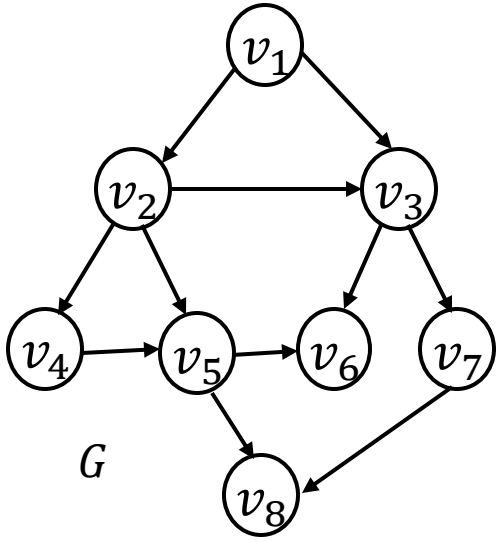
G

Stack empty, DFS from source v_1 finishes

$S =$

Practice

- Given the following graph G, show the process of the DFS if the source is v_2
  - You may use 0 to denote the color to be white, 1 to denote the color to be gray, and 2 to denote the color to be black



DFS: implementation

Algorithm 1: ***DFS(V, E, s)***

```
1  color← initialize an array of size n with all zero values
2  // Use 0 : white, 1 : gray, and 2: black
3  S ← an empty stack
4  S.push(s)
5  color[s] ← 1
6  while !S.isEmpty()
7     v ← S.top()
8     if v still has white-neighbor u
9            S.push(u)
10           color[u]=1
11    else
12       color[v] ← 2
13       S.pop()
14 Free color array if necessary
```

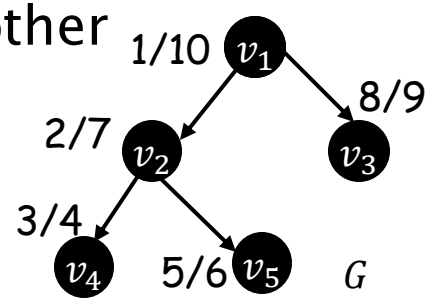
DFS: complexity analysis

- When a node v gets popped from the stack?
  - None of its out-neighbors is a white node
  - We may need to repeatedly check if node v has white out-neighbor
    - We need to check $d_{out}(v)$ times in the worst case
    - Cost: $d_{out}(v) \cdot d_{out}(v)$?
    - Can we do better?
      - We record the position checked last time
      - All nodes in previous positions will not be white
      - Cost: $O(d_{out}(v))$
- As each node is popped at most once, DFS's time cost is
  - $\sum_{u \in V} c(1 + d_{out}(u)) = c(n + m) = O(n + m)$

Properties of DFS

- Let $u.d$ and $u.f$ to indicate their first discovery time and their finish time, respectively, and denote $I(u)$ as the interval $[u.d, u.f]$

- We will only have three cases for two nodes u and v
  - $I(u) \subset I(v)$, u is the descendant of v
  - $I(v) \subset I(u)$, v is the descendant of u
  - $I(u) \cap I(v) = \emptyset$, neither one is the descendant of the other
  - Example:
    - $I(v_2)$: [2,7], I(v_4) = [3,4]
      - v_4 is a descendant of v_2 in the DFS tree



- We can check if a node u is a descendant of another node v in $O(1)$ time
  - If there is no such property, we need to retrieve the path